

SF-619 NF-14 · Modelo de datos canónico — investigación 2026-05-05

Fuente: análisis directo sobre BD producción (89 tablas, mageam.com) ejecutado 2026-05-05.

TL;DR

De las 7 dimensiones críticas, **5 ya tienen fuente única** (no necesitan trabajo). **2 están fragmentadas** y requieren consolidación: **materiales** (alta) y **costos** (media).

Dimensión	Estado	Acción
Ubicaciones técnicas	✓ única (<code>hierarchy_nodes</code> , 10168 filas)	nada
Equipos principales	✓ única (<code>hierarchy_nodes</code> + <code>node_type=EQUIPMENT</code>)	nada
Equipos de apoyo	✓ única (<code>support_equipment</code> , 128 filas)	nada
Workforce / mantenedores	✓ única (<code>workforce</code> , 35 filas)	nada
Work centers / planning groups	✓ única (<code>work_centers</code> , 27 filas)	nada
Materiales	● 3 tablas paralelas con IDs distintos	endpoint canónico ✓ implementado · fase 2: tabla mapping
Costos	● 7 tablas <code>annual_budget_*</code> por dimensión	aceptable, vista materializada para reporting

● Dimensión "materiales" — fragmentación real

Las 3 fuentes (sin foreign keys entre sí)

Tabla	Filas	ID	Propósito	Ejemplo
<code>bom_items</code>	2,304	<code>sap_code</code> (varchar)	BOM por equipo (componentes que tiene el equipo X)	<code>(P-1201A, "IMP-P1201", "11000100", "Impeller SS316")</code>
<code>inventory_items</code>	8,409	<code>material_code</code> (varchar)	Stock real por bodega	<code>("S26-MAT- 00001", warehouse_id=... , qty_on_hand=23)</code>
<code>sap_materials</code>	11,458	<code>sap_id</code> (varchar)	Catálogo maestro SAP global	<code>("000000010001" , "Rodamiento radial SKF 23248", unit_cost=18500)</code>

Problema observado

- **3 IDs distintos** (`sap_code` , `material_code` , `sap_id`) sin tabla de mapping.
- Los rodamientos SKF probablemente existen en las 3 tablas con códigos diferentes, sin forma trivial de cruzarlos.
- Cuando una OT necesita "rodamiento", no se sabe si el `bom_items.sap_code` de la planilla SAP del equipo es el mismo `inventory_items.material_code` que tiene la bodega.

Solución implementada (Sprint 6, hoy)

`GET /api/v1/sprint6/canonical/materials-search?query=rodamiento` busca en las 3 fuentes y devuelve resultados con prefijo `SAP:` / `INV:` / `BOM:` en `canonical_id` . Permite ver de un solo query qué hay en cada fuente.

Solución completa (próxima iteración)

Crear tabla `materials_canonical` :

```

CREATE TABLE materials_canonical (
  canonical_id VARCHAR(50) PRIMARY KEY,      -- p.ej. "MAT-CAN-00001"
  description VARCHAR(255),
  category VARCHAR(50),
  unit VARCHAR(20),
  -- Mapping a las 3 fuentes existentes
  sap_id VARCHAR(50) REFERENCES sap_materials(sap_id),
  inventory_material_code VARCHAR(50),      -- soft FK a inventory_items.material_code
  bom_sap_code VARCHAR(50),                -- soft FK a bom_items.sap_code
  -- Fingerprint para detectar duplicados textualmente
  description_fingerprint VARCHAR(80) INDEX, -- lowercase + sin diacríticos + tokens ordenados
  confidence_score FLOAT,                  -- qué tan seguros estamos del match (1.0 = manual, 0.7 =
  IA fuzzy)
  matched_at DATETIME,
  matched_by VARCHAR(50)                  -- 'manual', 'ai_fuzzy', 'sap_sync'
);

```

Plan de migración: 1. Generar fingerprint para las 3 tablas (script offline 1×). 2. Match exact: descriptions normalizadas iguales → 1 row canonical. 3. Match fuzzy IA (Claude): para no-matches, aplicar embedding o LLM con prompt "¿Son el mismo material? sí/no/dudoso". 4. Manual review: dudosos → cola de revisión humana. 5. Generar canonical_id para singletons (1 fuente = 1 canonical).

Endpoint downstream esperado:

```

GET /api/v1/canonical/materials/{canonical_id}
Returns:
{
  canonical_id: "MAT-CAN-00001",
  description: "...",
  sources: {
    sap: { sap_id: "...", unit_cost_usd: ..., manufacturer: ... },
    inventory: [ { warehouse_id, qty_on_hand, qty_available }, ... ],
    bom: [ { equipment_tag, quantity_per_unit, critical }, ... ]
  },
  derived: {
    total_stock: <sum inventory.qty_available>,
    applicable_equipment_count: <count bom>,
    avg_consumption_per_month: <calculado>
  }
}

```

● Dimensión "costos" — fragmentación aceptable

7 tablas `annual_budget_*` por dimensión analítica

Tabla	Filas	Eje analítico
<code>annual_budget_capex</code>	26	Inversiones
<code>annual_budget_equipment</code>	4,311	Por equipo
<code>annual_budget_executive</code>	31	Resumen ejecutivo
<code>annual_budget_kpi_targets</code>	39	Targets KPI
<code>annual_budget_maintenance</code>	225	Por estrategia mant
<code>annual_budget_opex</code>	147	Operacional
<code>annual_budget_production</code>	18	Producción
<code>cost_centers</code>	10	Centros de costo (raíz)

Análisis: la fragmentación aquí es **por dimensión analítica** (CAPEX vs OPEX vs equipment), no duplicación. Es estándar en EAM/ERP y aceptable.

Recomendación: vista materializada SQL `vw_budget_summary` que une por `equipment_tag` o `cost_center_id` para reporting consolidado, sin migrar tablas.

```
CREATE VIEW vw_budget_summary AS
SELECT
    eq.equipment_tag,
    SUM(CASE WHEN cap.equipment_tag = eq.equipment_tag THEN cap.amount END) AS capex,
    SUM(CASE WHEN op.equipment_tag = eq.equipment_tag THEN op.amount END) AS opex,
    SUM(CASE WHEN m.equipment_tag = eq.equipment_tag THEN m.amount END) AS maintenance,
    SUM(annual_budget_equipment.budget_total) AS equipment_total
FROM (SELECT DISTINCT equipment_tag FROM annual_budget_equipment) eq
LEFT JOIN annual_budget_capex cap ON cap.equipment_tag = eq.equipment_tag
LEFT JOIN annual_budget_opex op ON op.equipment_tag = eq.equipment_tag
LEFT JOIN annual_budget_maintenance m ON m.equipment_tag = eq.equipment_tag
LEFT JOIN annual_budget_equipment USING (equipment_tag)
GROUP BY eq.equipment_tag;
```

✓ Dimensiones ya unificadas

`hierarchy_nodes` (10168 filas) — fuente única para 6 niveles

Cubre **plant** → **área** → **system** → **equipment** → **sub-assembly** → **maintainable item**. Filtros por `node_type` (`EQUIPMENT` , `AREA` , `SYSTEM` , etc).

Este modelo está **bien diseñado** y es el *backbone* del sistema. No tocar.

`support_equipment` , `workforce` , `work_centers`

Cada uno con su propia tabla, sin duplicaciones detectadas. Solo agregar FKs duros donde hoy hay JSON con tags sueltos (`managed_work_orders.support_equipment[]`).

Plan de gobernanza propuesto

1. **Naming convention canonical_id por dimensión:** - Equipos: `EQ-{plant}-{seq}` (ya existe en `hierarchy_nodes.tag`) - Materiales: `MAT-CAN-{seq}` (nuevo, post-mapping) - Costos: no requiere ID nuevo (vista derivada)
2. **Versionado:** agregar columna `version` + `valid_from/valid_to` a tablas canonicals para soportar slow-changing dimensions (ej: equipo cambia de área).
3. **API única:** `/api/v1/canonical/{dimension}/...` como contrato downstream.
4. **Source of truth doc:** este archivo + diagrama ER en `docs/canonical-er.md` (pendiente).

Esfuerzo estimado para iteración 2

- Tabla `materials_canonical` + script de migración + UI revisión: **~3-4 días dev** + 1 día QA con datos reales.
- Vista `vw_budget_summary` : **~2h** + tests.
- Naming convention + versionado: **~1 día** decisional + implementación incremental.
- API `/canonical/*` endpoints downstream: **~2 días**.

Total iteración 2: ~1 sprint (5-7 días) para llevar materiales a canonical real.